

Traffic Intelligence System Smart Traffic & Emergency Portal

Report submitted as part of the internship program requirement for the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE ENGINEERING (AI & ML)

**Submitted by:
ALLIPILLI ROHIT (A24126552067)**



DEPARTMENT OF CSE (AI & ML)

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
(UGC AUTONOMOUS)**

**(Permanently Affiliated to AU, Approved by AICTE, and Accredited by NBA &
NAAC)**

**SANGIVALASA, BHEEMILI MANDAL, VISAKHAPATNAM - 531162
2025-2026**

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND
SCIENCES**

(Affiliated to Andhra University)

SANGIVALASA, VISAKHAPATNAM - 531162

2025-2026



BONAFIDE CERTIFICATE

This is to certify that this Case Study Report titled “TRAFFIC INTELLIGENCE SYSTEM” from APSSDC-Summer Online Internship is the bonafide work of ALLIPILLI ROHIT (A24126552067) of III/IV CSE (AI & ML), carried out under the guidance and supervision of the Department of CSE (AI & ML), Anil Neerukonda Institute of Technology and Sciences.

REVIEWER

Mrs J.A.Lavanya
Assistant Professor
Department of CSE (AI & ML)
ANITS

CLASS TEACHER

P Santosh Kumar
Assistant Professor
Department of CSE (AI & ML)
ANITS

HEAD OF THE DEPARTMENT

DR. K. SELVANI DEEPTHI

Department of CSE (AI & ML)
ANITS

ACKNOWLEDGEMENT

The successful completion of a project is the result of continuous learning, guidance, encouragement, and support from many individuals. I take this opportunity to express my sincere gratitude to everyone who contributed to the successful completion of this case study project.

First and foremost, I express my sincere gratitude to Dr. K. S. Deepthi, Head of the Department of Computer Science & Engineering (AI & ML), ANITS, for providing the necessary support, guidance, and encouragement throughout the development of this project.

I would also like to express my heartfelt thanks to my faculty members and project mentors, particularly my reviewer Mrs J.A.Lavanya, for their valuable suggestions, technical guidance, and continuous encouragement during the different stages of the project. Their feedback helped me improve my understanding of machine learning and its practical applications in predictive modeling.

I am grateful to the Department of CSE (AI & ML), Anil Neerukonda Institute of Technology and Sciences, for providing the academic environment and resources required to carry out this project successfully.

I would also like to thank my classmates and friends for their support, discussions, and suggestions during the development and testing of the project.

Finally, I express my sincere gratitude to everyone who directly or indirectly contributed to the successful completion of the "Traffic Intelligence System" project.

ALLIPILLI ROHIT
(A24126552067)

Table of Contents

1. Introduction
 - 1.1 Background of the Internship / Project
 - 1.2 Objectives of the Internship
 - 1.3 Importance of Full Stack ML Development Skills
 - 1.4 Scope of the Project
2. Organization Profile
 - 2.1 About APSSDC
 - 2.2 Program Structure and Mentorship Model
3. Project Overview
 - 3.1 Title of the Project
 - 3.2 Problem Statement
 - 3.3 Objectives of the Project
 - 3.4 Relevance and Real-World Use Case
4. Literature Review / Theoretical Background
 - 4.1 Version Control and Collaborative Development
 - 4.2 Front-End Fundamentals (HTML, CSS, JavaScript)
 - 4.3 Back-End Development with Python and Flask
 - 4.4 Data Preprocessing (Pandas and NumPy)
 - 4.5 Machine Learning Algorithms (Regression)
 - 4.6 Predictive Modeling for Traffic and Risk
 - 4.7 REST API Design and Client-Server Architecture
 - 4.8 Local Hosting and Deployment
5. Methodology / Development Roadmap
 - 5.1 Phase 1 — Front-End Foundations
 - 5.2 Phase 2 — Backend API and Data Modelling
 - 5.3 Phase 3 — ML Model Training
 - 5.4 Phase 4 — System Integration
 - 5.5 Phase 5 — Model Serialization and Deployment
6. Implementation
 - 6.1 Week-by-Week Implementation Schedule
 - 6.2 Backend Implementation — Traffic API
 - 6.3 Frontend Implementation
 - 6.4 Traffic Prediction Model Integration
7. Results and Discussion
 - 7.1 Features Delivered

- 7.2 API Endpoint Summary
- 7.3 Application Walkthrough
- 7.4 Security Measures and Current Gaps

8. Conclusion & Future Work

- 8.1 Summary of Learning
- 8.2 Skills Acquired
- 8.3 Limitations of the Current Approach
- 8.4 Future Scope

9. Internship / Project Outcomes

- 9.1 Technical Skills Acquired
- 9.2 Soft Skills Developed
- 9.3 Contribution to Career Growth

10. Source Code and Project Structure

- 10.1 Project Directory Structure
- 10.2 ML Backend Source Code
- 10.3 Frontend Source Code
- 10.4 API Deployment Code

Appendix

- A. Source Code (GitHub Repository Link)
- B. Application Screenshots
- C. Certificate of Completion

1. Introduction

1.1 Background of the Internship / Project

This report presents the design, development and outcomes of the Traffic Intelligence System, a full stack web application that combines traffic volume prediction and risk analysis. The project was undertaken as part of an APSSDC-Summer Online Internship program in fulfilment of the requirements of the Bachelor of Technology in Computer Science and Engineering (AI & ML).

The system was built end-to-end — from data generation and machine learning implementation using Scikit-Learn to a responsive front end connected via a Python Flask API, integrating predictive modeling into a practical interface.

1.2 Objectives of the Internship

- To gain hands-on experience in designing and implementing a REST API using Python and Flask.
- To model and train datasets using Pandas and Scikit-Learn.
- To build a responsive front end using HTML, CSS and vanilla JavaScript, without a complex framework.
- To implement end-to-end data pipelines for real-time inference.
- To deploy a full stack ML application locally.

1.3 Importance of Full Stack ML Development Skills

Full stack development skills allow an engineer to reason about and contribute to an application end-to-end — from what a user sees and interacts with, to how their data is processed, analyzed, and served. This breadth shortens the feedback loop between an idea and a working product, and enables debugging across the client, network and server rather than being confined to a single layer.

1.4 Scope of the Project

The scope of the Traffic Intelligence System was to design and build a multi-page web application that allows a user to input environmental factors (time, temperature, weather, road quality) and instantly receive predictions for expected traffic volume and accident risk through an integrated ML model.

2. Organization Profile

2.1 About APSSDC

The Andhra Pradesh State Skill Development Corporation (APSSDC) served as the training and mentorship organization that structured and delivered this Machine Learning

internship. The program followed a project-based, mentor-guided model pairing hands-on coding with deep theoretical concepts.

2.2 Program Structure and Mentorship Model

The program consisted of structured sequential sessions covering Python, Data Preprocessing, Machine Learning algorithms, and Deployment. The capstone project was an independent full stack web application designed to demonstrate applied competence rather than only completion of exercises.

3. Project Overview

3.1 Title of the Project

Traffic Intelligence System — Smart Traffic & Emergency Portal (Built with Python, Scikit-Learn, Flask and vanilla JavaScript).

3.2 Problem Statement

Urban planning and emergency response systems lack real-time predictive analytics based on weather and road conditions. Commuters and city planners need a unified platform to forecast traffic volumes and evaluate accident risk dynamically without switching between disconnected platforms.

3.3 Objectives of the Project

- Build a predictive model capable of recognizing risk patterns.
- Provide a user-friendly HTML interface to collect environmental inputs.
- Implement a Flask REST API to serve serialized model predictions.
- Connect the frontend directly to the backend for seamless JSON parsing.

3.4 Relevance and Real-World Use Case

The project mirrors the shape of production data science platforms: a REST API backed by a machine learning model, a client application that consumes that API over HTTP, and a decoupled inference engine. This pattern is widely used in modern applications spanning recommendation engines and autonomous routing systems.

4. Literature Review / Theoretical Background

4.1 Version Control and Collaborative Development

Git is a distributed version control system that tracks changes to source code over time, while GitHub provides remote hosting and collaboration tooling on top of it. The codebase is organized as a single Git repository.

4.2 Front-End Fundamentals (HTML, CSS, JavaScript)

HTML provides the semantic structure, CSS governs its visual presentation, and JavaScript adds interactivity by communicating with the backend API using the Fetch API. The project uses vanilla JavaScript for optimal performance.

4.3 Back-End Development with Python and Flask

Flask is a minimal, unopinionated web framework layered on top of Python that structures an application around routes and middleware. It is highly suited for serving ML models because it shares the Python runtime with Scikit-Learn.

4.4 Data Preprocessing (Pandas and NumPy)

Pandas and NumPy are used to formulate the synthetic datasets. We mapped environmental conditions such as 'Snow' and 'Poor Roads' to heavily penalize risk scores, allowing the algorithm to correctly infer relationships during training.

4.5 Machine Learning Algorithms (Regression)

Linear Regression was utilized to predict continuous output variables (Traffic Volume). Logistic Regression was used for binary output classification (Accident Risk: High/Low) by analyzing feature correlation.

4.6 Predictive Modeling for Traffic and Risk

A text classifier is typically used for spam, but here we used regression for physics-based prediction. The system captures heuristics like time-of-day rush hours to adjust volume, combining rules with learned coefficients.

4.7 REST API Design and Client-Server Architecture

REST structures an API around endpoints manipulated through standard HTTP verbs (POST). The frontend is a decoupled client calling a single endpoint, mimicking mobile app consumption.

4.8 Local Hosting and Deployment

The Flask server is hosted on a local port (5000), while the HTML frontend is served statically, providing real-time local network deployment capabilities.

5. Methodology / Development Roadmap

5.1 Phase 1 — Front-End Foundations

Static page structure and styling for the core index page using HTML and CSS, defining the layout, color palette (Blue and Tangerine), and input dropdown elements.

5.2 Phase 2 — Backend API and Data Modelling

Initialization of the Flask server, routing definitions, and JSON body-parsing mechanisms to prepare for the connection with the ML models.

5.3 Phase 3 — ML Model Training

Generation of 50,000 logical synthetic rows. Training Linear and Logistic regression models and exporting them using the joblib library for persistence.

5.4 Phase 4 — System Integration

Wiring the frontend Fetch API to the backend Flask `/predict` route, ensuring data serialization across the stack and handling CORS (Cross-Origin Resource Sharing).

5.5 Phase 5 — Model Serialization and Deployment

Testing edge cases (e.g. Snow + Poor Road) to ensure the system accurately flags high-risk situations, followed by finalizing code structures.

6. Implementation

6.1 Week-by-Week Implementation Schedule

Week 1: Requirement analysis, Python basics, and static front-end pages.
Week 2: Flask server setup, Data Generation scripts.
Week 3: Pandas manipulation, Linear/Logistic regression training.
Week 4: API routing and joblib model integration.
Week 5: Frontend Fetch integration, DOM manipulation, and dynamic result rendering.
Week 6: Testing, styling polish, and report generation.

6.2 Backend Implementation — Traffic API

The Flask application is initialized with CORS and parses JSON bodies. The `/predict` endpoint receives environmental inputs, converts them to a Pandas DataFrame, and runs `model.predict()`.

6.3 Frontend Implementation

A single HTML file utilizes an `analyzeTraffic()` asynchronous JS function to gather values by element ID, push them via POST, and modify the `innerText` of the result boxes based on JSON output.

6.4 Traffic Prediction Model Integration

The `.pkl` files act as serialized memory. By loading them globally when the server starts, the API achieves near-zero latency because it does not need to retrain the 50,000 rows on every request.

7. Results and Discussion

7.1 Features Delivered

- Searchable predictive analytics interface.
- Continuous volume prediction.
- Binary accident risk classification.
- Decoupled Flask REST API.
- Model persistence via joblib.
- Responsive UI styled consistently.

7.2 API Endpoint Summary

POST /predict : Accepts environmental parameters and returns calculated traffic volume and risk classifications.

7.3 Application Walkthrough

A user visits the frontend, selects the hour, temperature, weather, weekend status, and road quality, then clicks the action button. The result pane dynamically appears, displaying outputs in standard text or red-highlighted 'HIGH RISK' warnings.

7.4 Security Measures and Current Gaps

CORS is enabled globally. However, the system currently lacks authentication, and relies on client-side state. Future implementations would require JWT tokens for secured enterprise access.

8. Conclusion & Future Work

8.1 Summary of Learning

Building this system provided end-to-end practical experience across the full stack: designing ML pipelines, exposing REST routes through Flask, building a vanilla-JavaScript frontend, and linking the two seamlessly.

8.2 Skills Acquired

Acquired skills spanning DOM manipulation, Fetch API, Flask routing, Pandas dataframe manipulation, Scikit-Learn training algorithms, and REST conventions.

8.3 Limitations of the Current Approach

The data is synthetic rather than pulled from live IoT traffic sensors. Additionally, regression models lack the temporal awareness that deep learning sequence models (like LSTMs) provide.

8.4 Future Scope

Future work involves integrating real-time Google Maps API data, deploying the backend to Render, and upgrading the prediction algorithm to a Neural Network for complex pattern recognition.

9. Internship / Project Outcomes

9.1 Technical Skills Acquired

- Hands-on experience designing and building a REST API.
- Practical model training with Scikit-Learn.
- Front-end development without a framework.
- Applied machine learning: training a text classifier and serving it.

9.2 Soft Skills Developed

- Improved debugging skills by working across the browser, API, and Python layers.
- Strengthened planning and time-management.
- Enhanced technical communication through structured documentation.

9.3 Contribution to Career Growth

Produced a complete, deployable full-stack project suitable for a professional portfolio, demonstrating competence across the entire software development lifecycle.

10. Source Code and Project Structure

10.1 Project Directory Structure

traffic-intelligence/



10.2 ML Backend Source Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
import joblib
import os
```

```

# Generate synthetic traffic data
np.random.seed(42)
n_samples = 50000

data = {
    'hour': np.random.randint(0, 24, n_samples),
    'temperature': np.random.randint(-10, 40, n_samples),
    'weather_code': np.random.randint(0, 3, n_samples),
    'is_weekend': np.random.randint(0, 2, n_samples),
    'road_quality': np.random.randint(0, 3, n_samples)
}
df = pd.DataFrame(data)

df['volume'] = (500 + (df['hour']*20) - (df['is_weekend']*200) -
(df['weather_code']*150) + (df['road_quality']*100) + np.random.normal(0, 50,
n_samples)).astype(int)
df['volume'] = df['volume'].clip(lower=0)

risk_score = (df['weather_code']*0.4) - (df['road_quality']*0.3) +
np.random.normal(0, 0.2, n_samples)
df['risk'] = (risk_score > 0.3).astype(int)

X = df[['hour', 'temperature', 'weather_code', 'is_weekend', 'road_quality']]
y_vol = df['volume']
y_risk = df['risk']

vol_model = LinearRegression().fit(X, y_vol)
risk_model = LogisticRegression(max_iter=1000).fit(X, y_risk)

os.makedirs('models', exist_ok=True)
joblib.dump(vol_model, 'models/volume_model.pkl')
joblib.dump(risk_model, 'models/risk_model.pkl')

```

10.3 Frontend Source Code

```

<!-- index.html snippet -->
<button onclick="analyzeTraffic()">Predict Traffic & Risk</button>
<script>
    async function analyzeTraffic() {
        const data = {
            hour: document.getElementById('hour').value,
            temperature: document.getElementById('temperature').value,
            weather_code: document.getElementById('weather_code').value,
            is_weekend: document.getElementById('is_weekend').value,
            road_quality: document.getElementById('road_quality').value
        };
        const response = await fetch('http://127.0.0.1:5000/predict', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(data)
        });
        const result = await response.json();
        if(result.success) {
            document.getElementById('out-vol').innerText =
result.traffic_volume;

```

```

        document.getElementById('out-risk').innerText =
result.accident_risk == 1 ? "HIGH RISK" : "LOW RISK";
    }
}
</script>

```

10.4 API Deployment Code

```

from flask import Flask, request, jsonify
from flask_cors import CORS
import pandas as pd
import joblib

app = Flask(__name__)
CORS(app)
vol_model = joblib.load('models/volume_model.pkl')
risk_model = joblib.load('models/risk_model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json
    input_data = pd.DataFrame([data])
    v = vol_model.predict(input_data)[0]
    r = risk_model.predict(input_data)[0]
    return jsonify({'success': True, 'traffic_volume': int(v), 'accident_risk':
int(r)})

if __name__ == '__main__':
    app.run(port=5000)

```

Appendix A. Source Code

Source code available at: <https://github.com/rohit6758/traffic-accident>

Appendix B. Application Screenshots

```
C:\Users\rohit>cd C:\Users\rohit\gemini\antigravity\scratch\traffic-accident

C:\Users\rohit\gemini\antigravity\scratch\traffic-accident>python train_models.py
Models trained and saved successfully in traffic-intelligence/models/

C:\Users\rohit\gemini\antigravity\scratch\traffic-accident>python server.py
Starting Flask API Server on http://localhost:5000
  * Serving Flask app 'server'
  * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
  * Running on http://127.0.0.1:5000
Press CTRL+C to quit
  * Restarting with watchdog (windowsapi)
Starting Flask API Server on http://localhost:5000
  * Debugger is active!
  * Debugger PIN: 261-414-540
_
```

Traffic Intelligence

Time of Day (Hour: 0-23)

14

Temperature (°C)

25

Weather Condition

Clear



Is it the Weekend?

No (Weekday)



Road Quality

Good



Predict Traffic & Risk

14

Temperature (°C)

25

Weather Condition

Clear

Is it the Weekend?

No (Weekday)

Road Quality

Good

Predict Traffic & Risk

Prediction Results:

 Expected Traffic Volume: **1,596** vehicles

 Accident Risk: **LOW RISK**

Appendix C. Certificate of Completion

	ANDHRA PRADESH STATE SKILL DEVELOPMENT CORPORATION DEPARTMENT OF SKILLS DEVELOPMENT & TRAINING GOVERNMENT OF ANDHRA PRADESH	
CERTIFICATE OF COMPLETION		
CERTIFICATE NO: APSSDC/SIP/2026-27/30422		
This is to certify that <u>Allipilli Rohit</u> , bearing Roll Number <u>A24126552067</u> , from		
<u>Anil Neerukonda Institute Of Technology and Sciences</u> , has successfully completed the course		
<u>Machine Learning using Python</u> under the APSSDC-Summer Online Internship -2026 conducted by the Andhra		
Pradesh State Skill Development Corporation (APSSDC) from <u>04-May-2026</u> to <u>03-Jul-2026</u> .		
Signed by: G. Ganesh Kumar (IAS) (Managing Director) TimeComp: 2026-07-27 14:03:18 IST 		
MD & CEO (G. GANESH KUMAR, I.A.S.)		